Le Mans Université

Licence Informatique *2ème année*

Module 174UP02 Rapport de Projet

JurassicWar

Solène Orieux, Romane Saint-Léger, Hannah Sergent

5 avril 2026

Table des matières

1 Introduction **3**
1.1 Contexte 3
1.2 Énoncé du problème 3
1.3 Objectifs initiaux du projet 3

2 Conception **3**
2.1 Principales règles du jeu 3
2.2 Foncionnalités prévues 3

3 Oraganisation du travail **3**
3.1 Répartition des tâches 3
3.2 Planning prévisionnel 3
3.3 Outils de programmation utilisés 3

4 Developpement **3**
4.1 Représentation du terrain par une matrice 3
4.2 Positionnement et mobilité des dinosaures 3
4.3 Système d'actions et outils à disposition du joueur 4
4.3.1 Mécanique des changements de tours 4
4.3.2 Infliger un coup mortel : bombe 4
4.3.3 Blesser un dinosaure : armes de tir 6
4.3.4 Autre méthode de déplacement : grapin 6
4.4 Interface graphique et utilisateur 7
4.4.1 Utilisation du module SDL 7
4.4.2 Création d'un menu principal 7
4.4.3 Faciliter l'utilisation du jeu pour l'utilisateur 7

5 Résultats et Conclusion **7**

6 Bibliographie **7**

7 Annexes **8**
7.1 Jeux de tests 8
7.2 Traces de débogage 8
7.2.1 Correction du système de collisions de la bombe 8

1 Introduction

1.1 Contexte

1.2 Énoncé du problème

1.3 Objectifs initiaux du projet

2 Conception

2.1 Principales règles du jeu

2.2 Foncionnalités prévues

3 Oraganisation du travail

3.1 Répartition des tâches

3.2 Planning prévisionnel

3.3 Outils de programmation utilisés

4 Developpement

4.1 Représentation du terrain par une matrice

4.2 Positionnement et mobilité des dinosaures

4.3 Système d'actions et outils à disposition du joueur

Cette partie rend compte de la manière dont s'enchaînent des actions dans JurassicWar à travers la mécanique des changements de tours mais aussi de la nature des actions possibles dans le jeu. Un dinosaure peut en effet user de différents outils tels que la bombe, les armes de tirs ou encore le grapin.

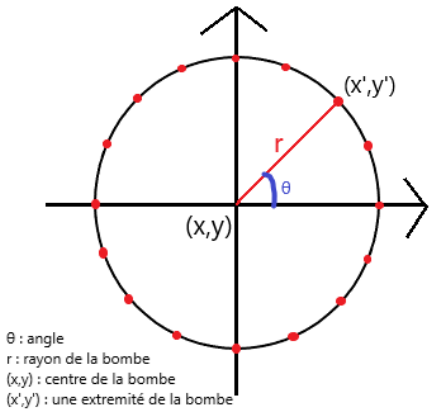
4.3.1 Mécanique des changements de tours

Premièrement, les dinosaures jouent chacun leur tour, c'est-à-dire que l'utilisateur n'a pas la possibilité de choisir avec lequel de ses trois dinosaures il souhaite agir à un tour donné. L'instauration de cette règle augmente d'une part la complexité du jeu, mais d'autre part le système des changements de tours, puisque l'alternance entre tous les dinosaures doit être correcte même lorsqu'un ou plusieurs d'entre eux meurent. Afin de mettre en œuvre les changements de tours, une fonction *tourSuivant* a été crée. Elle s'appuie sur l'identifiant associé à chaque dinosaure pour déterminer le dinosaure qui peut jouer. Cependant, comme les dinosaures de la première équipe sont numérotés de un à trois et ceux de la seconde équipe de quatre à six, à la suite du premier dinosaure, c'est au quatrième dinosaure de jouer et non pas au second dinosaure pour garantir l'alternance entre les deux équipes. Ceci nécessite de mémoriser deux identifiants : celui du dinosaure qui vient d'agir et celui du dinosaure ayant agi précédemment. Ces identifiants sont stockés dans uns structure *t_ tour* qui contient également le numéro de l'équipe qui vient de jouer et un compteur du nombre de tours effectués. Par ailleurs, la fonction *tourSuivant* doit tenir compte de l'état vivant ou mort d'un dinosaure dans la mesure où un dinosaure mort ne peut plus jouer. Ainsi, une fois que la fonction a choisi un dinosaure, elle vérifie qu'il est vivant. Dans le cas contraire, elle sélectionne un autre dinosaure vivant appartenant au même joueur.

4.3.2 Infliger un coup mortel : bombe

Deuxièmement, au cours d'une partie, les joueurs peuvent faire appel à une arme mortelle : la bombe. Contrairement aux autres armes évoquées dans la section suivante, la bombe tue instantanément le dinosaure qu'elle heurte. Néanmoins, cette arme ne peut être utilisée uniquement si le dinosaure dispose d'une place suffisante à côté de lui pour la positionner. Par ailleurs, cette arme se caractérise également par sa capacité à rebondir sur le terrain. Cette capacité de la bombe à rebondir induit l'utilisation de plusieurs notions mathématiques dans le code. Tout d'abord, la mise en place des rebonds nécessite un système de collision avec le terrain efficace. La bombe ayant une forme circulaire, la fonction de collision avec le terrain nommée *collisionTerrainBombe* fait recourt aux coordonnées polaires pour analyser

les points sur le contour de la bombe. Les points analysés sont représentés en rouge sur la figure 1 et sont au nombre de trente-deux. En cas de collision



$$x' = x + r\cos(\theta)$$

$$y' = y + r\sin(\theta)$$

FIGURE 1 – Schéma des coordonnées polaires

avérée entre un de ces points et la terre, la pente du terrain est étudiée pour que la bombe rebondisse perpendiculairement

4.3.3 Blesser un dinosaure : armes de tir

4.3.4 Autre méthode de déplacement : grapin

4.4 Interface graphique et utilisateur

4.4.1 Utilisation du module SDL

4.4.2 Création d'un menu principal

4.4.3 Faciliter l'utilisation du jeu pour l'utilisateur

5 Résultats et Conclusion

6 Bibliographie

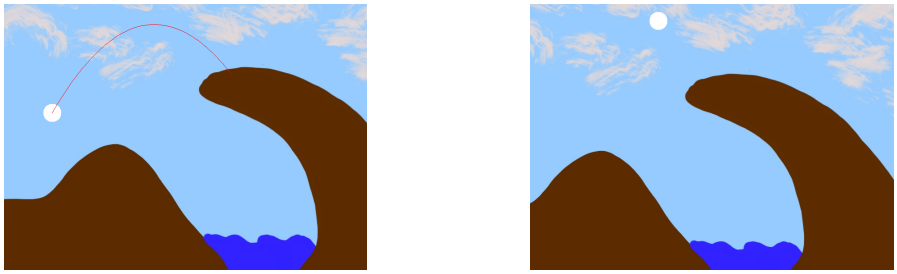
7 Annexes

7.1 Jeux de tests

7.2 Traces de débogage

7.2.1 Correction du système de collisions de la bombe

Lors de la création des fonctions chargées de détecter un contact entre la bombe et le terrain, la mer ainsi que les extrémités de la fenêtre, un bug silencieux a été rencontré, necessitant l'usage de l'outil gdb pour identifier le dysfonctionnement. La situation est la suivante :



(a) Trajectoire prévue pour la bombe (b) Arrêt anormal de la bombe

FIGURE 2 – Illustration du parcours incorrect effectué par la bombe

La bombe interrompt sa trajectoire au milieu du ciel avant d'avoir heurté la terre. Il s'agit par conséquent d'identifier une erreur dans une des fonctions liées à la bombe, en particulier dans la fonction *collisionTerrainBombe*. Pour cela, les fichiers impliqués dans l'affichage et le système de collisions sont compilés avec l'option -g :

```
CC = gcc

FLAGS_COMPILATION = -Wall -g `sdl2-config --cflags`
FLAGS_EDITION = `sdl2-config --libs` -lSDL2_image -lSDL2_ttf

PROG = ./bombe

OBJETS = \
    testFonctionsRebonds.o \
    fonctionsRebonds.o \
    fonctionsVerification.o \
    chargerMatrice.o

all: $(PROG)

$(PROG): $(OBJETS)
    $(CC) $(OBJETS) $(FLAGS_EDITION) -o $(PROG)

%.o: %.c
    $(CC) $(FLAGS_COMPILATION) -c $< -o $@
```

FIGURE 3 – Extrait du makefile utilisé pour le débogage des collisions

Les lignes qui suivent constituent la séquence des instructions de débogage mise en oeuvre pour corriger le système de collision. Cette séquence de commande nécessite de provoquer artificiellement l'évènement clavier déclenchant le lancer de la bombe afin d'analyser le contenu des variables lorsque l'arme est en mouvement puis lors de la collision. Dans le jeu, c'est la touche b du clavier, représentée par la macro SDL_SCANCODE_B dans le module SDL qui amorce le tir de la bombe. Cette macro possédant pour valeur cinq, l'élément d'indice cinq du tableau *etatClavier* qui consigne les touches actionnées par l'utilisateur est par conséquent positionné à un au cours de l'exécution du programme sous gdb.

Séquence des instructions de débogage :

```
gdb bombe
# Mise en place des points d'arrêt
# Initialiser l'affichage des variables
(gdb) break testFonctionsRebonds.c :67
# Simuler l'appui sur la touche b du clavier
(gdb) break testFonctionsRebonds.c :92
# Vérifier le contenu des variables au départ du lancement de la bombe
(gdb) break testFonctionsRebonds.c :98
# Analyser les valeurs des variables à la suite d'une collision avec le terrain
(gdb) break testFonctionsRebonds.c: 117
# Provoquer un arrêt en cas de collision avec le terrain
(gdb) break fonctionsRebonds.c:60 \
if ((matriceTerrain[bombe->coor.x][bombe->coor.y + bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x][bombe->coor.y - bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x + bombe->rayon][bombe->coor.y] == TERRE) \
|| (matriceTerrain[bombe->coor.x - bombe->rayon][bombe->coor.y] == TERRE))
# Lancement de l'exécution du programme
(gdb) run
(gdb) display bombeLancee
bombeLancee = 0
(gdb) display bombe.coor
bombe.coor = {x = 325, y = 350}
(gdb) continue
bombeLancee = 0
bombe.coor = {x = 325, y = 350}
(gdb) set etatClavier[5] = 1
(gdb) continue
bombeLancee = 1
bombe.coor = {x = 325, y = 350}
(gdb) continue
Thread 1 "bombe" hit Breakpoint 5, \
```

```
rebondTerrainBombe (matriceTerrain=0x7fffffc85290, bombe=0x7fffffc85234) \
at fonctionsRebonds.c:60 \
return ((matriceTerrain[bombe->coor.x][bombe->coor.y + bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x][bombe->coor.y - bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x + bombe->rayon][bombe->coor.y] == TERRE) \
|| (matriceTerrain[bombe->coor.x - bombe->rayon][bombe->coor.y] == TERRE));
(gdb) continue
bombeLancee = 1
bombe.coor = {x = 592, y = 131}
(gdb) next
bombeLancee = 0
bombe.coor = {x = 592, y = 131}
(gdb) quit
```

À l'issue de la séquence de commandes, il s'avère que l'erreur réside dans une inversion de coordonnées dans la fonction *collisionTerrainBombe*, comme l'illustre l'arrêt au point d'arrêt numéro cinq. Cette inversion s'explique par une différence de convention entre la matrice et l'affichage dans la fenêtre avec le module SDL. Tandis que la largeur correspond à l'axe des ordonnées pour la matrice, il s'agit de l'axe des abscisses pour l'affichage et inversement pour la hauteur. La résolution du comportement incohérent de la bombe nécessite donc uniquement d'échanger les coordonnées x et y dans les tests effectués par la fonction.